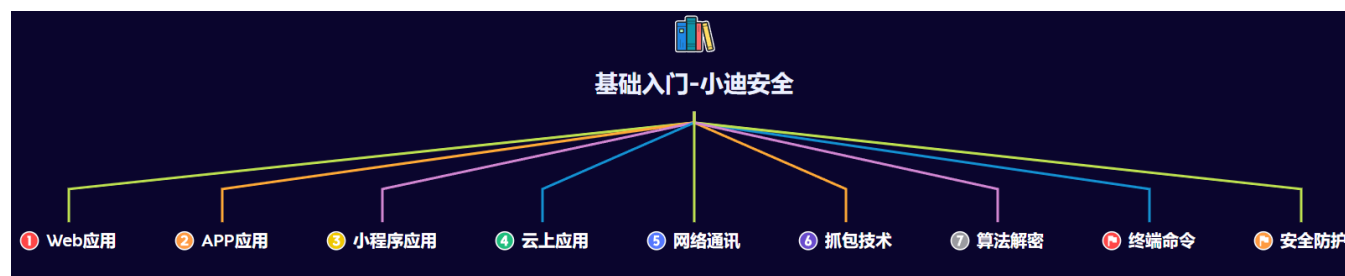


Day13 基础入门-网站协议&身份鉴权&OAuth2安全&Token令牌&JWT值&Authirization标头



1.知识点

- 1、基础入门-Web应用-域名上的技术要点
- 2、基础入门-Web应用-源码上的技术要点
- 3、基础入门-Web应用-数据上的技术要点
- 4、基础入门-Web应用-解析上的技术要点

- 1、基础入门-Web应用-搭建架构上的技术要点
- 2、基础入门-Web应用-源码类别上的技术要点

- 1、基础入门-Web应用-防护产品-WAF保护
- 2、基础入门-Web应用-加速服务-CDN节点
- 3、基础入门-Web应用-文件托管-OSS存储
- 4、基础入门-Web应用-通讯服务-反向代理
- 5、基础入门-Web应用-运维安全-负载均衡

- 1、基础入门-Web应用-蜜罐系统
- 2、基础入门-Web应用-堡垒机运维
- 3、基础入门-Web应用-内外API接口
- 4、基础入门-Web应用-第三方拓展架构

- 1、基础入门-APP应用-开发架构安全问题
- 2、基础入门-小程序应用-开发架构安全问题

- 1、抓包技术-Web应用-http/s-Burp&Yakit
- 2、抓包技术-APP应用-http/s-Burp&Yakit

- 3、抓包技术-PC端应用-http/s-Burp&Yakit
 - 4、抓包技术-WX小程序-http/s-Burp&Yakit
 - 5、抓包技术-软件联动-http/s-Proxifier
 - 6、抓包技术-通用方案-http/s-ReqableApi
 - 7、抓包技术-其他工具-http/s-Fiddler&Charles
-

- 1、抓包技术-HTTP/S双层代理-扶墙环境
 - 2、抓包技术-HTTP/S上游下游-项目联动
 - 3、抓包技术-全局协议-WireShark&科来
-

- 1、数据不回显原因和解决-带外延迟反弹写文件
 - 2、数据不出网原因和解决-出入站策略正反向连接
-

- 1、反弹Shell-项目&命令&语言等
 - 2、系统渗透命令-网络&文件&操作等
-

- 1、传输格式&数据-类型&编码&算法
 - 2、密码存储&混淆-不可逆&非对称性
-

- 0、算法类型-单向散列&对称性&非对称性
 - 1、算法识别加解密-MD5&AES&DES&RSA
 - 2、解密条件寻找-逻辑特征&源码中&JS分析
-

- 1、请求头&返回包-方法&头修改&状态码等
 - 2、数据包分析-红队攻击工具&蓝队流量研判
 - 3、数据包构造-Reqable自定义添加修改请求
-

- 1、网站协议-http/https安全差异（抓包）
 - 2、身份鉴权-HTTP头&OAuth2&JWT&Token
-

2.详细点

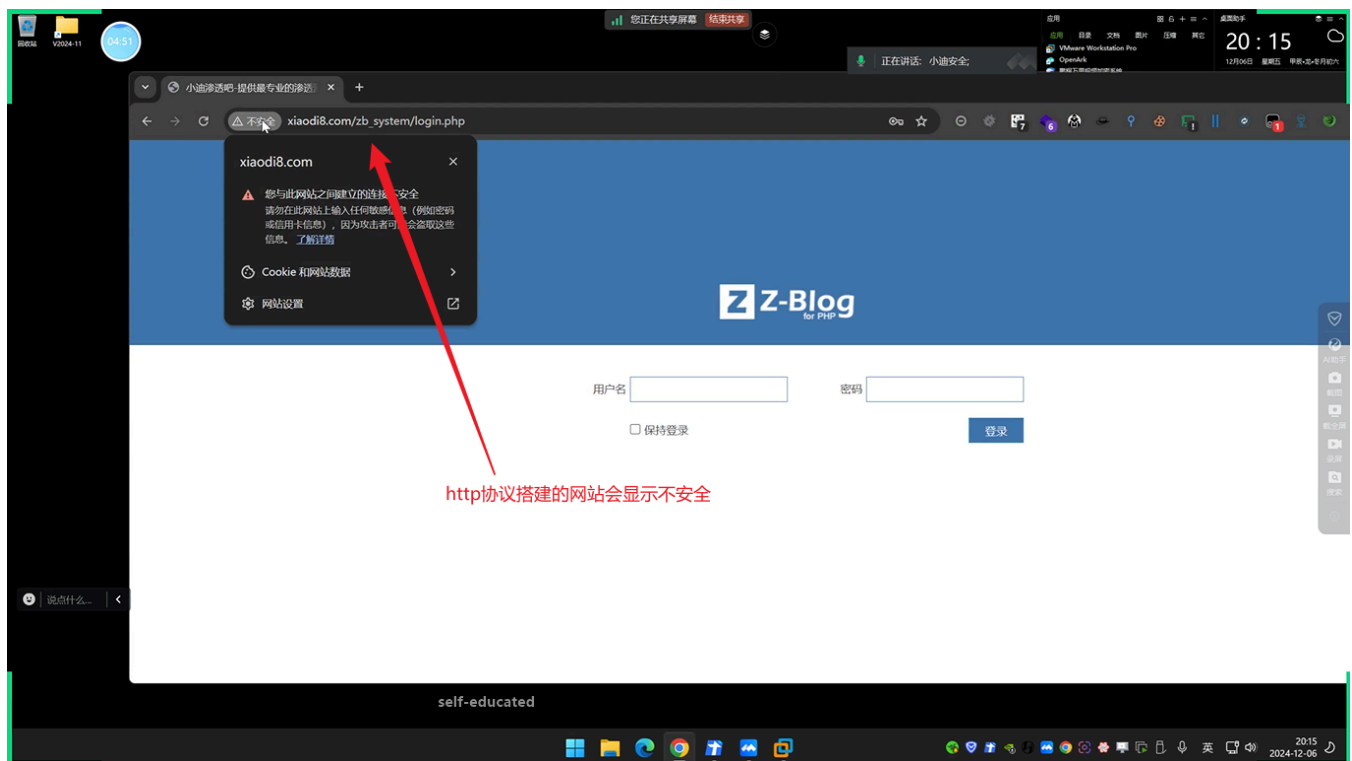
- 架构：WEB,APP,小程序,前后端,容器化等
- 服务：OSS存储,CDN加速,云数据库,负载均衡等
- 网络：不回显,反向代理,防火墙出入站,内外网,正反向连接等

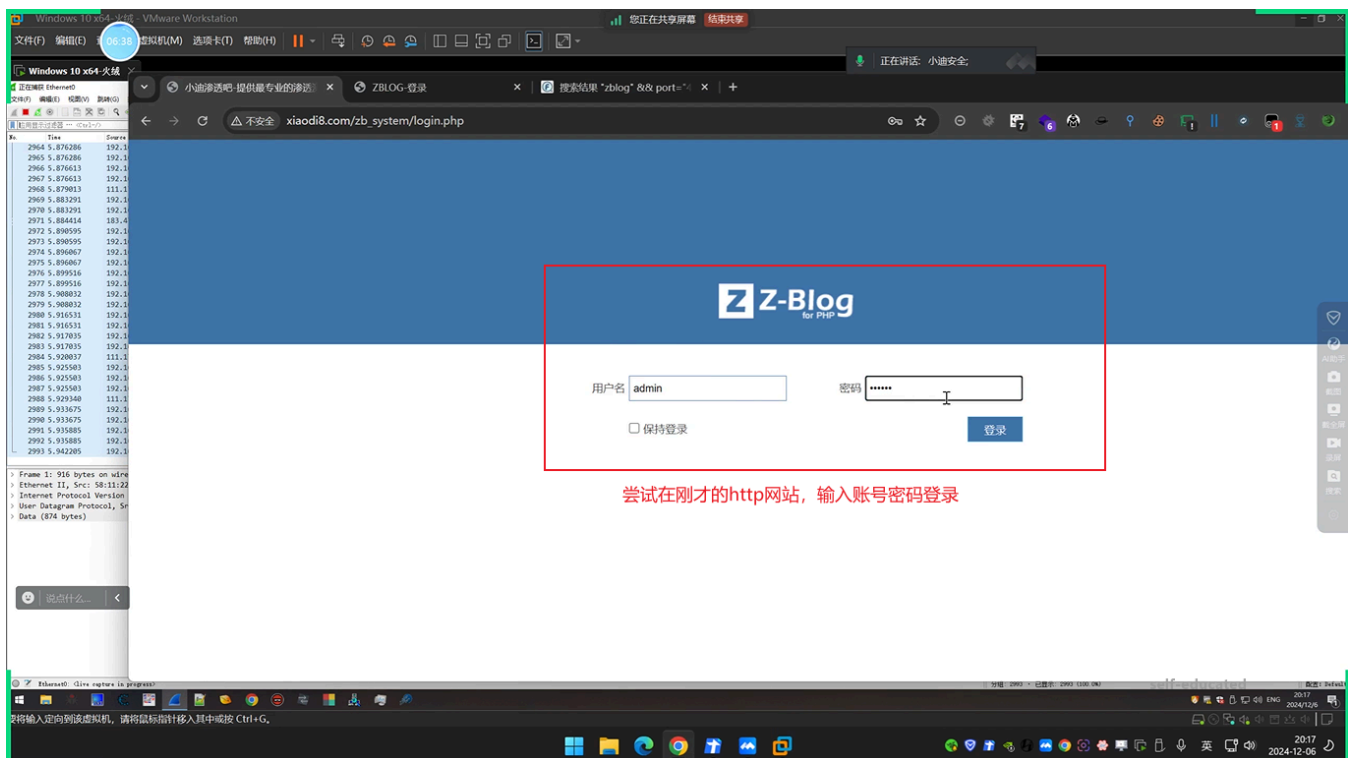
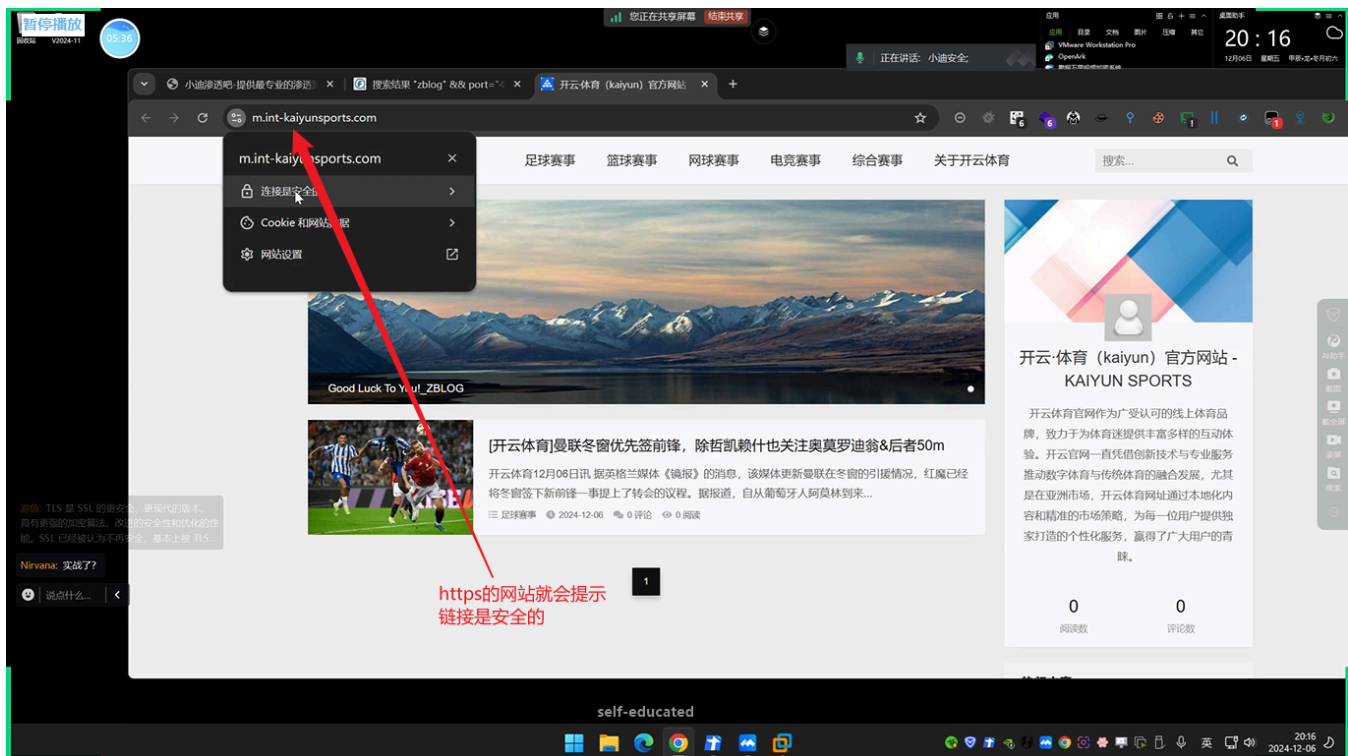
- 算法: MD5,Base64,AES,DES,Salt,自定义,代码加密算法等
- 命令: Windows&Linux,文件下载,网络查看,反弹权限,用户等
- 防护: WAF防护,蜜罐系统,CDN加速,权限设置,加壳加密反调试等

3.演示案例

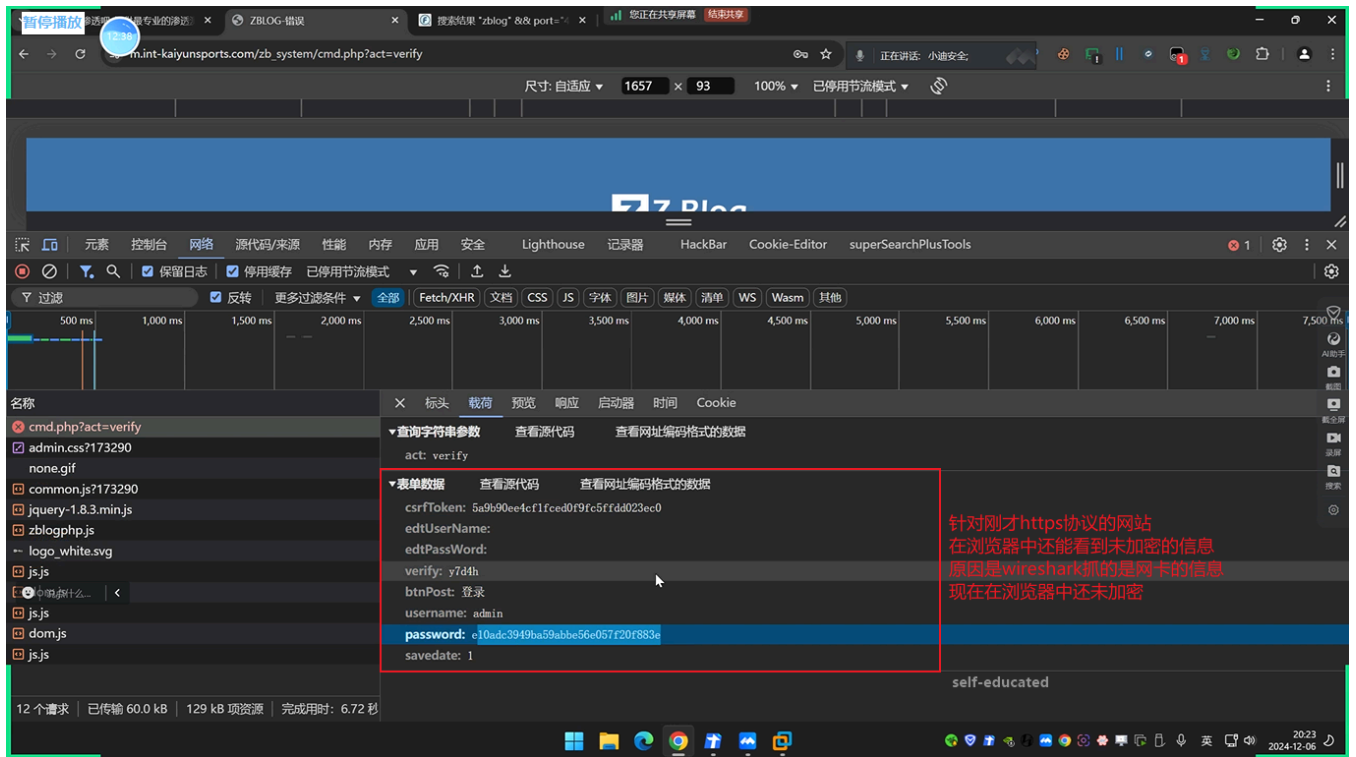
3.1 网站协议-http&https-安全测试差异性

1. 加密方式
- 2 HTTP: 使用明文传输,数据在传输过程中可以被任何人截获和查看。
- 3 HTTPS: 通过SSL/TLS协议对数据进行加密,确保数据在传输过程中不被第三方截获和篡改。
- 4 2. 身份验证
- 5 HTTP: 不需要进行身份验证,任何人都可以访问网站。
- 6 HTTPS: 通过数字证书和SSL/TLS协议验证服务器的身份,防止"中间人攻击"。
- 7 3. 端口号
- 8 HTTP: 默认使用80端口。
- 9 HTTPS: 默认使用443端口,提供更高的安全性。









3.2 身份鉴权- Authorization&Token&JWT&OAuth

1

Cookie, Session, Token, JWT, oauth2等

2

参考: https://mp.weixin.qq.com/s/Z6rt_ggCA8dNVJPgELZ44w

3

应用场景:

4

Cookie+Session简单, 建议在网内使用;

5

Token相对完善, 推荐在跨域外网使用;

6

JWT推荐使用, 常用在SSO单点登录中;

7

OAuth灵活方便, 对于第三方系统登录更友好。

3.2.1 Cookie+Session

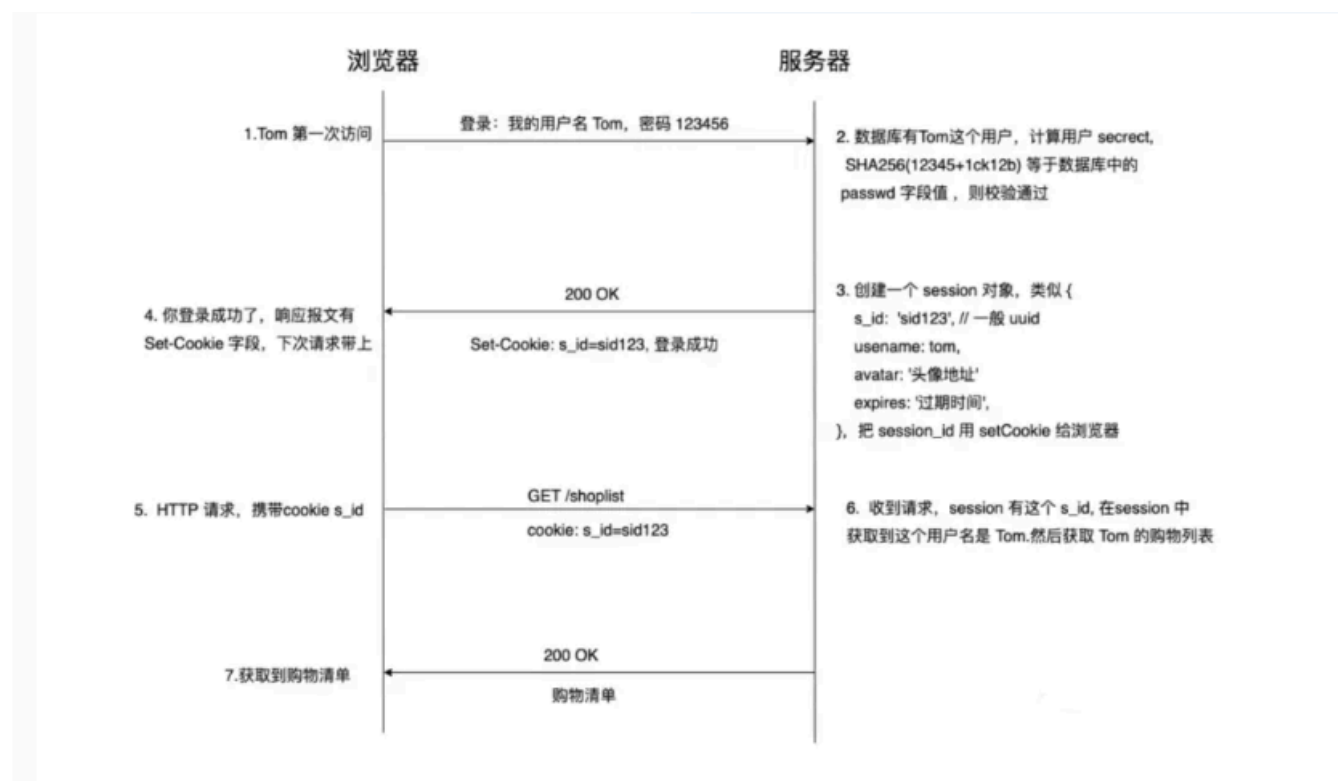
注册时, 服务器在接收到注册账号密码之后, 使用足够长且随机的salt值对密码进行加密, 来达到数据防泄露的目的。

注册流程如下:



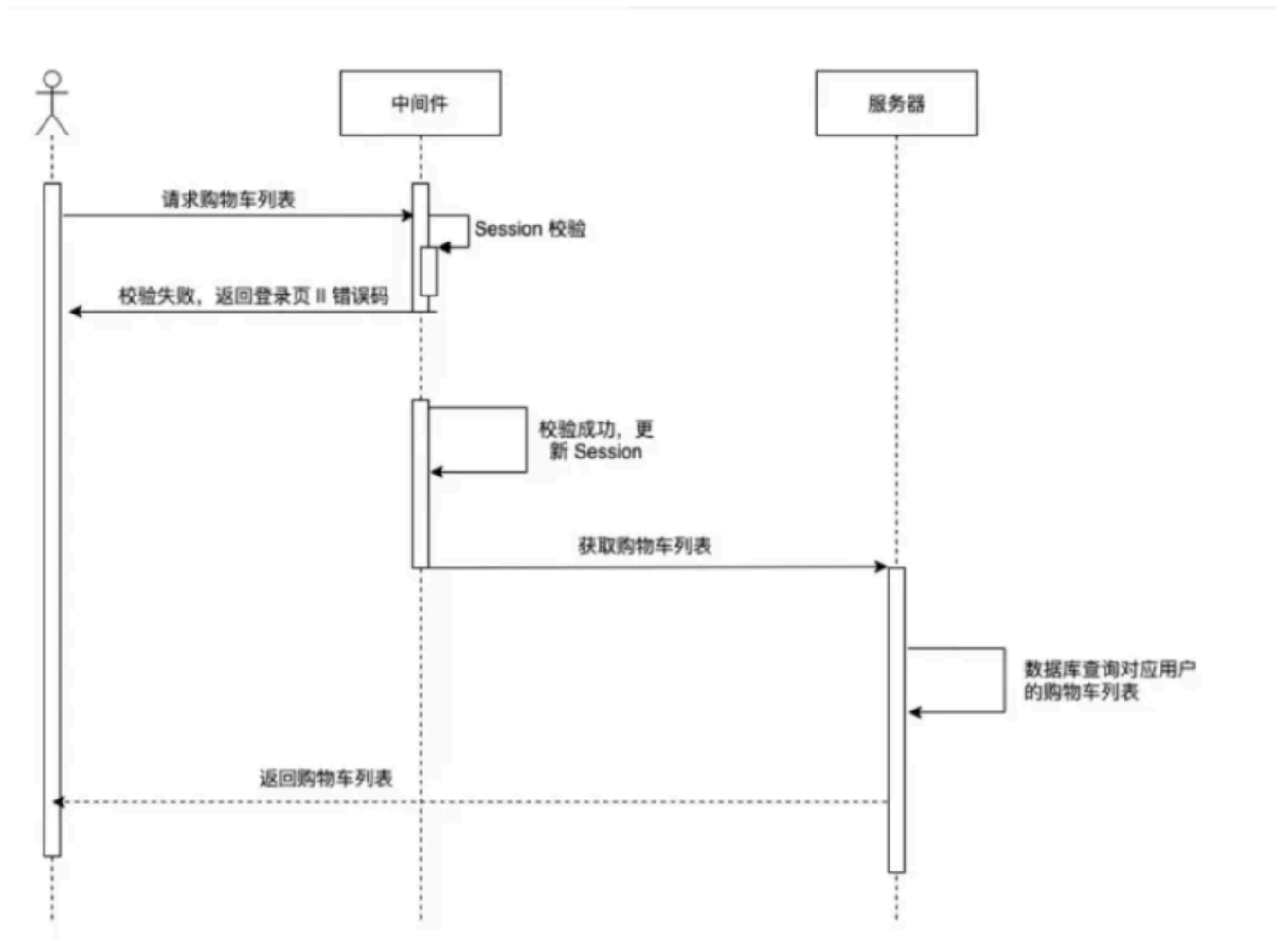
登录时，服务器通过salt值与密码进行SHA加密与数据库中的password进行比对，确定密码正确；创建一个session对象并把sessionId通过cookie的形式保存在浏览器中，浏览器发送请求时带上cookie值sessionId，服务器端通过解析sessionId获取用户身份，进行业务操作。

登录流程如下：



更新session操作：session的过期时间比较短，为了避免用户频繁登录，session的过期时间理应随着用户的操作延长，需要刷新session。以 Koa 为例，开发一个 middleware（默认情况下所有请求都会经过该 middleware），如果校验 Session 有效，就更新 Session 的 expires: 当前时间+过期时间。

更新session流程如下：

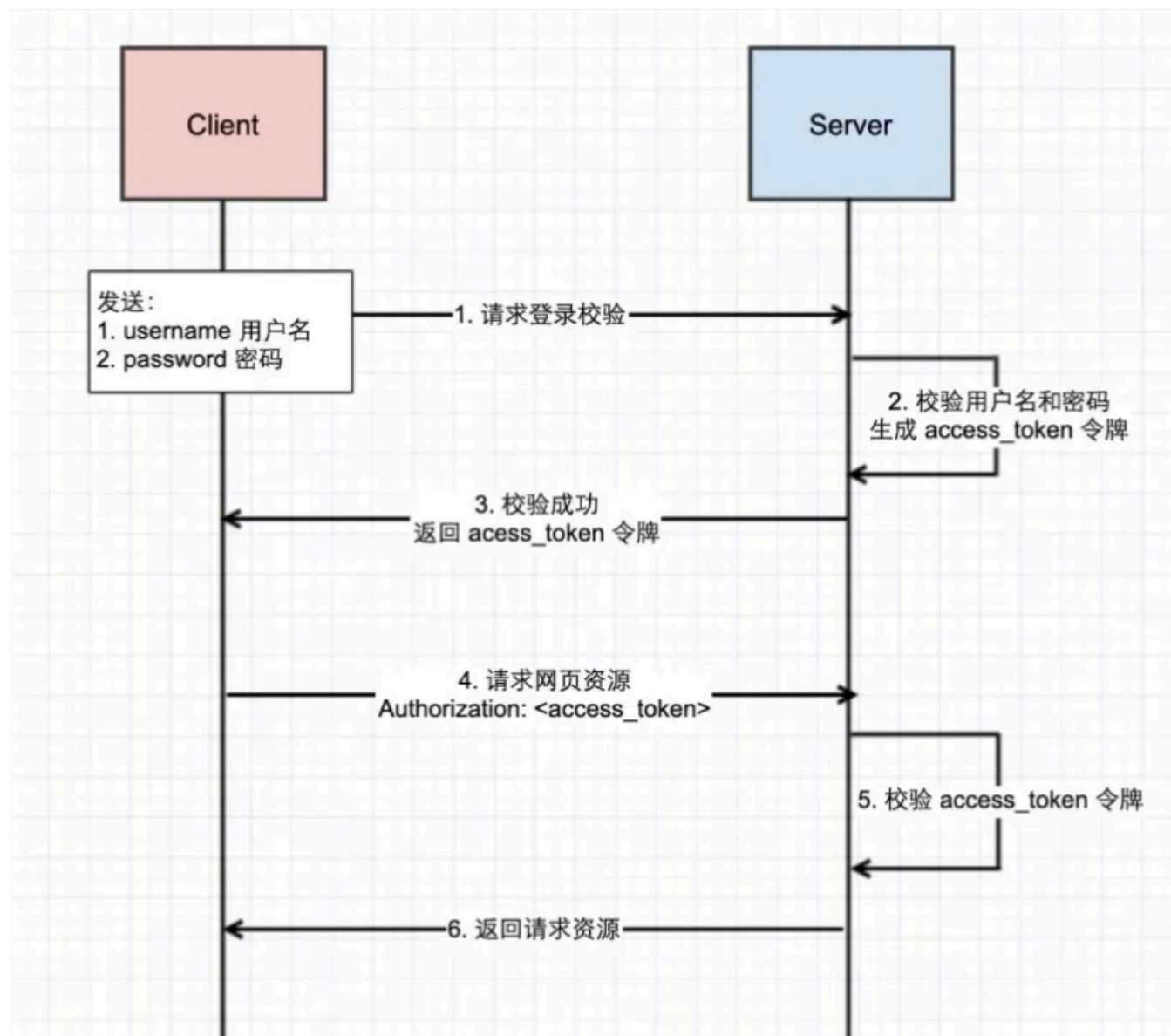


3.2.2 Token



- 1 Token，通常叫做令牌，是一种自定义实现的类似Session/Cookie机制的，用来代替传统Session/Cookie的新兴鉴权方案，当前很多的应用API鉴权就是使用的Token令牌。
- 2
- 3 Token是服务端生成的一串加密字符串，用户在登录成功后生成并返回给客户端，之后客户端的每次请求都会通过GET/POST/Header等方式携带Token，服务端通过验证Token的有效性来完成鉴权。
- 4
- 5 一般 Token 的组成：uid（用户唯一的身份标识）+ time（当前时间的时间戳）+ sign（签名，Token 的前几位以哈希算法压缩成的一定长度的十六进制字符串）
- 6
- 7 基于Token的鉴权方式的完整流程为：
- 8
- 9 (1) 客户端用户通过用户和密码进行首次登录；
- 10 (2) 服务端接收用户请求，并验证用户名和密码的正确性，登录验证成功后根据自定义规则生成Token信息；
- 11 (3) 服务端将生成的Token通过响应返回给客户端；
- 12 (4) 客户端将Token信息存储在本地；
- 13 (5) 客户端在之后的每次请求中携带Token信息；
- 14 (6) 服务端针获取请求中的Token，并根据定义的验证机制判断Token合法性，验证成功获取用户信息，保持用户状态；
- 15 (7) Token存活时间达到设置的有效期后自动失效，此后用户请求时Token验证不通过，需要用户重新登录验证。

token校验的流程如下：



3.2.3 JWT



- 1 JWT全称JSON web Token，是一个字符串书写规范，作用是用来在用户和服务器之间传递安全可靠的信息，它由三部分组成：头部、载荷、签名。



- 1 头部
- 2 头部用于描述关于该JWT的最基本的信息，例如其类型以及签名所用的算法等。这也可以被表示成一个JSON对象，下例中在头部指明了签名算法是HS256算法，然后对它进行base64编码，形成第一部分
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9
- 3 头部示例：{"typ":"JWT","alg":"HS256"}

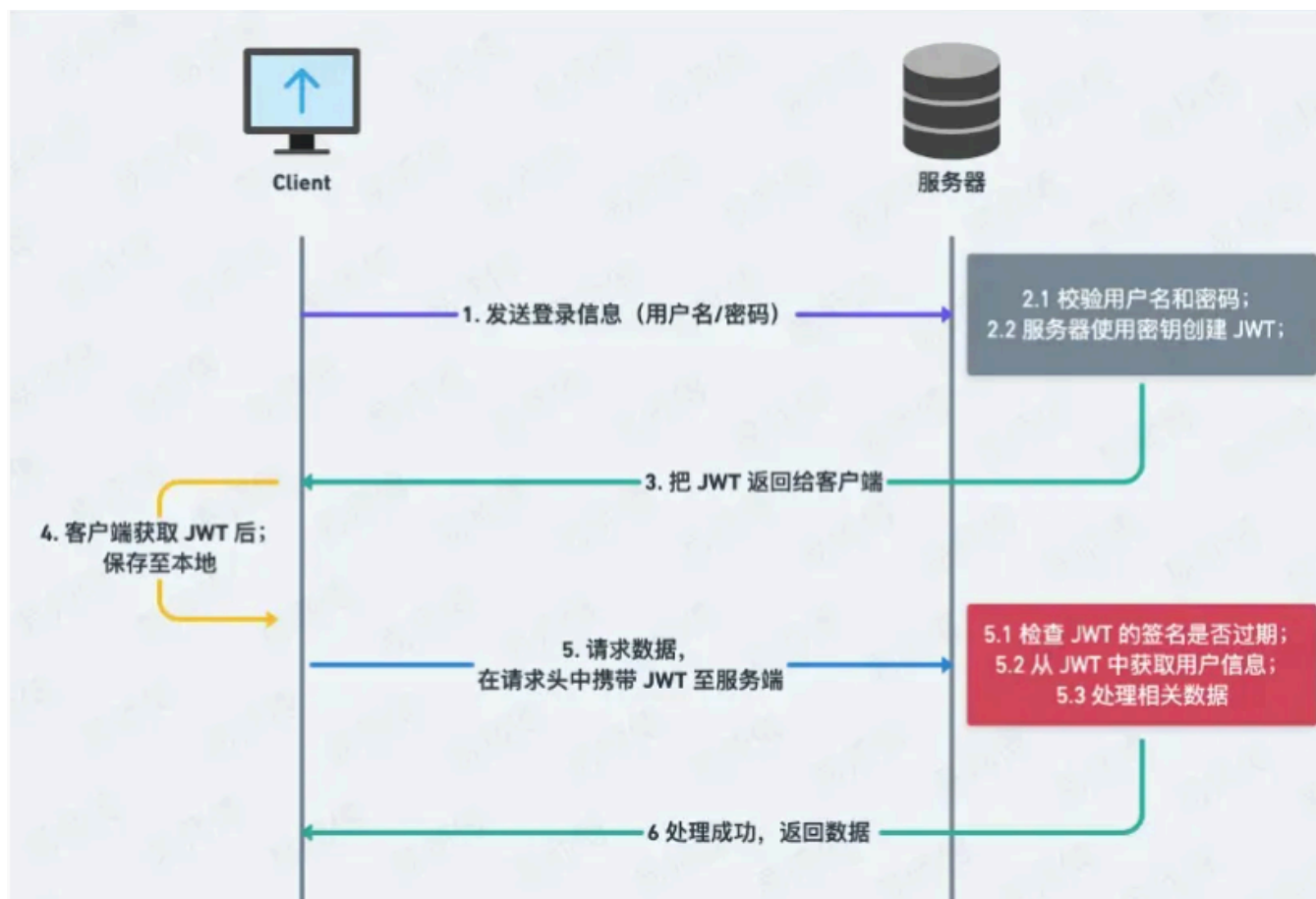


- 1 荷载 (payload)
- 2 荷载就是存放有效信息的地方，**payload**部分的数据可以分为三种类型：
 - 3 (1) **Registered claims** (注册的声明)
 - 4 这些是一组预定义的声明，写在**JWT**标准中，但并不是强制要求携带的。
 - 5 (2) **Public claims** (公共的声明)
 - 6 这些声明可以随意定义，通常用于存放一些用户**ID**、用户类型等非敏感信息，这些声明应该避免与注册声明的名称冲突。
 - 7 (3) **Private claims** (私有的声明)
 - 8 这些声明是自定义的，用于在**JWT**中存储一些特定的应用信息，这些声明应该使用不与**IETF JSON**中任何已定义的注册名称相冲突的**URI**作为名称。



- 1 签证 (signature)
- 2 **jwt**的第三部分是一个签证信息，这个签证信息由三部分组成：**header** (base64后的)、**payload**(base64后的)、**secret**。这个部分需要base64加密后的**header**和base64加密后的**payload**使用“.”连接组成的字符串，然后通过**header**中声明的加密方式进行**secret**组合加密，然后就构成了**jwt**的第三部分。

JWT鉴权流程如下：



3.2.4 OAuth



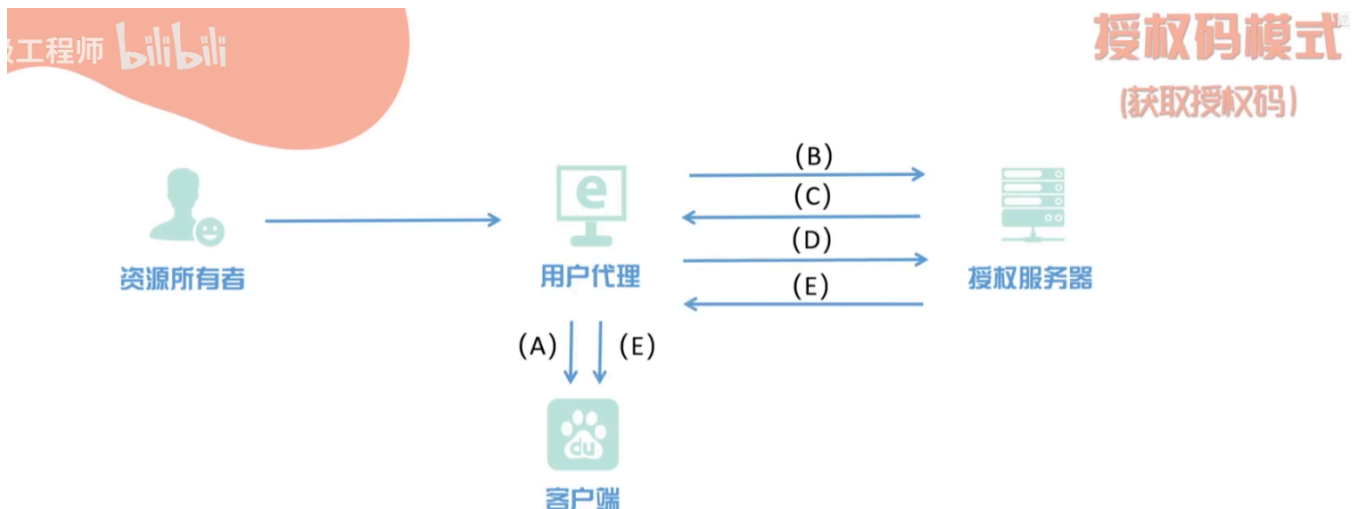
- 1 #OAuth2技术
- 2 授权框架，使网站和web应用程序能够请求对另一个应用上的用户帐户进行有限访问。至关重要的是，OAuth允许用户授予此访问权限，而无需向请求应用程序公开其登录凭据。这意味着用户可以微调他们想要共享的数据，而不必将其帐户的完全控制权移交给第三方。

(1) 验证模式：



- 1 四种验证模式：
- 2 `authorization_code` 授权码模式
- 3 `implicit code` 简单模式
- 4 `password` 密码模式
- 5 `client_credentials` 客户端模式

(2) 授权码模式流程：



(A) 用户通过代理（浏览器）访问客户端

(B) 客户端将用户导向授权服务器

(C) 授权服务器询问用户是否给予客户端授权

(D) 用户同意授权

(E) 授权服务器将用户导向客户端指定的"重定向URI"，并附上授权码

`/oauth/authorize?client_id=baidu&response_type=code&scope=all
&redirect_uri=http://www.baidu.com`

client_id

客户端id，需要在授权服务器中注册的

scope

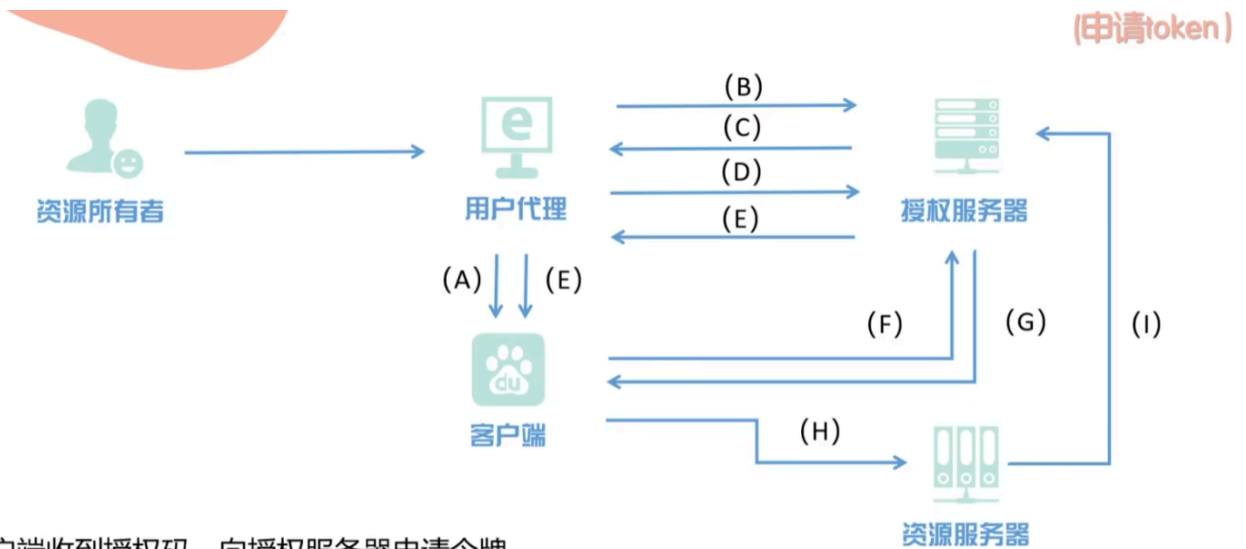
权限范围，在授权服务器配置

response_type

`code`返回授权码，`token`返回访问令牌

redirect_uri

重定向URI，用于授权成功后跳转

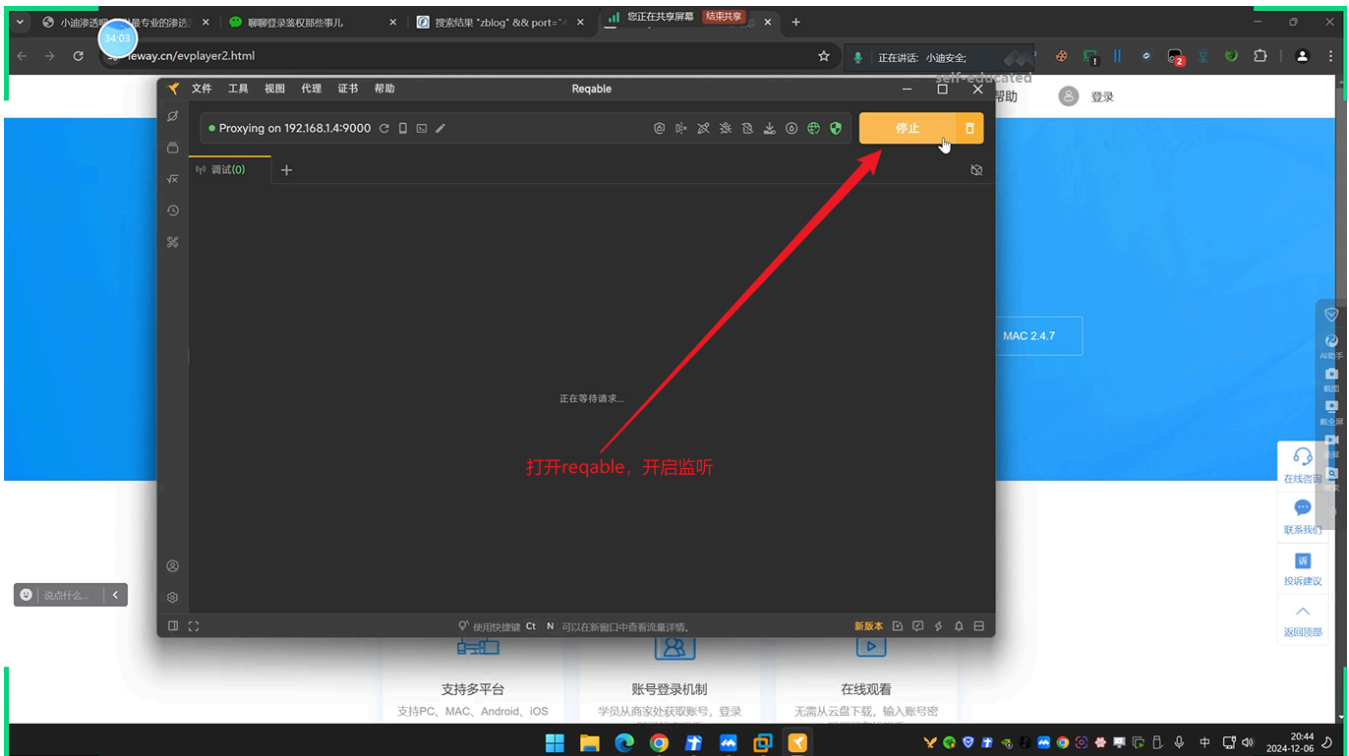


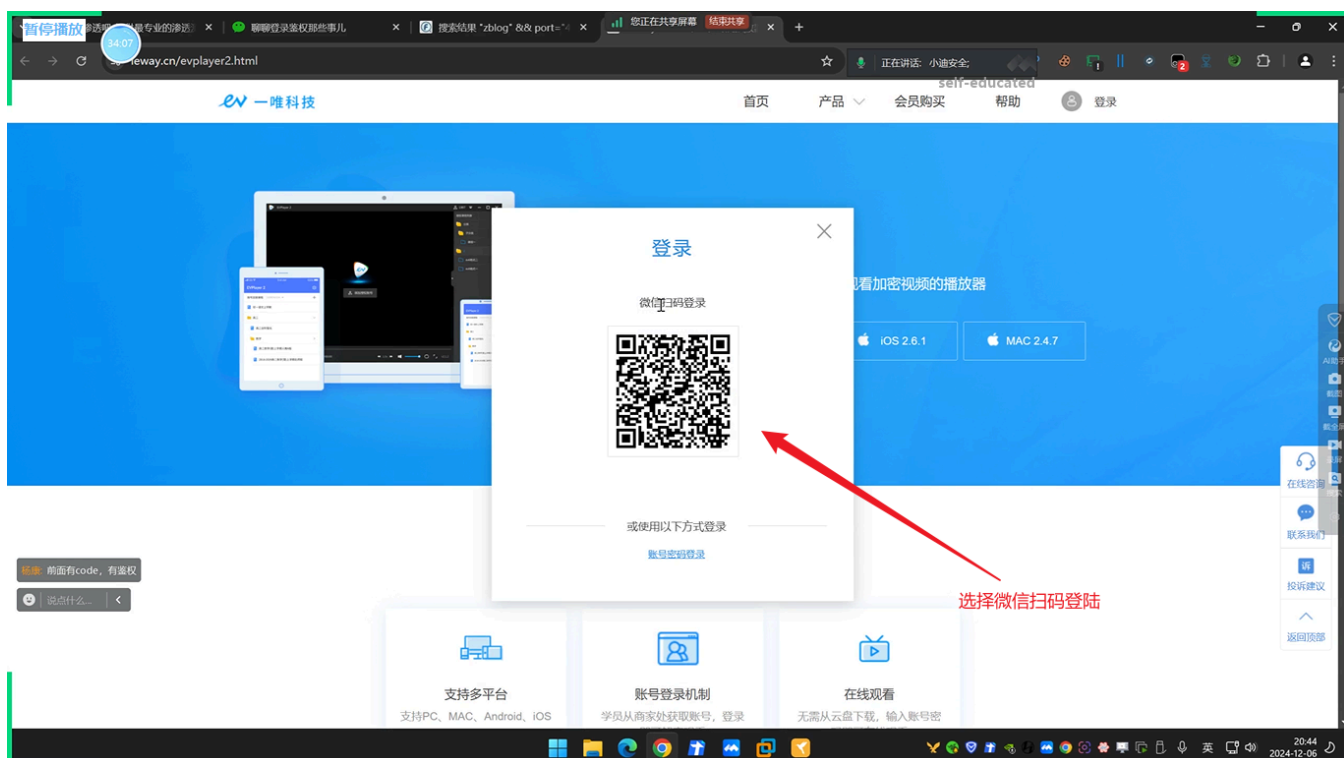
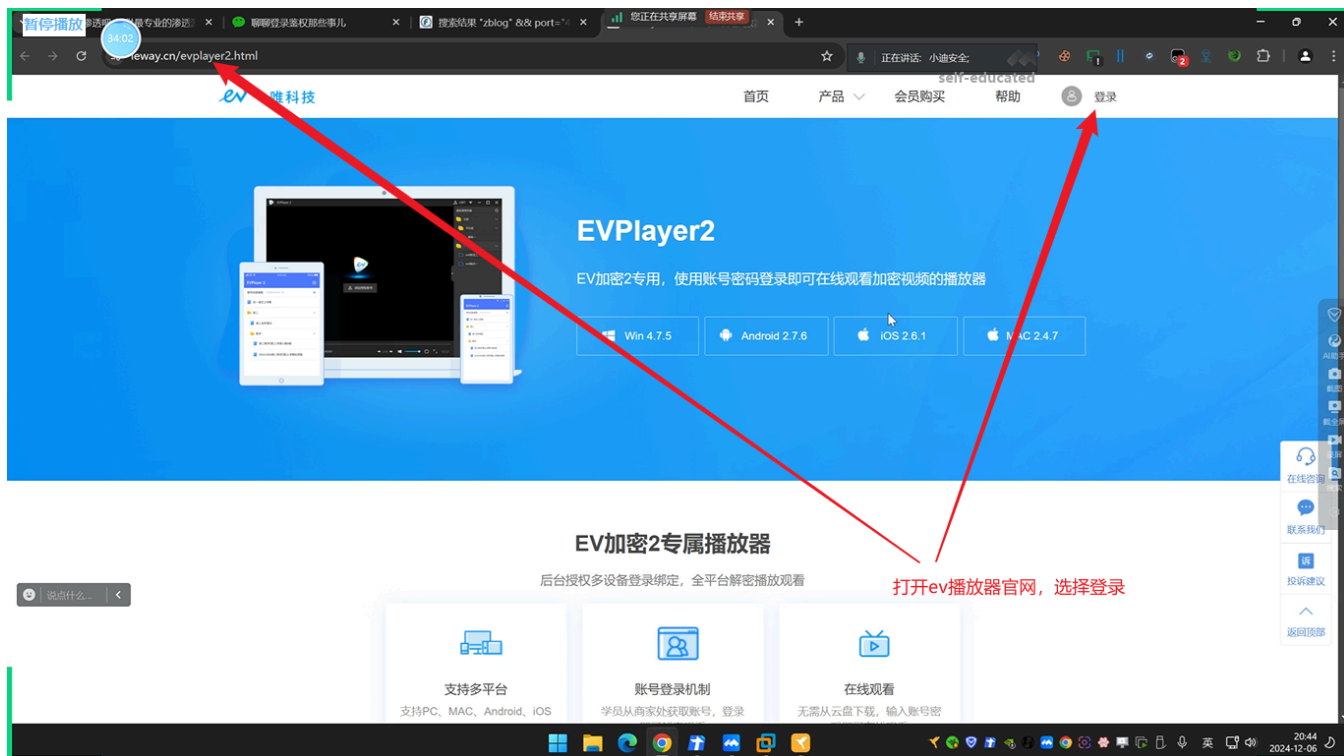
(F) 客户端收到授权码，向授权服务器申请令牌

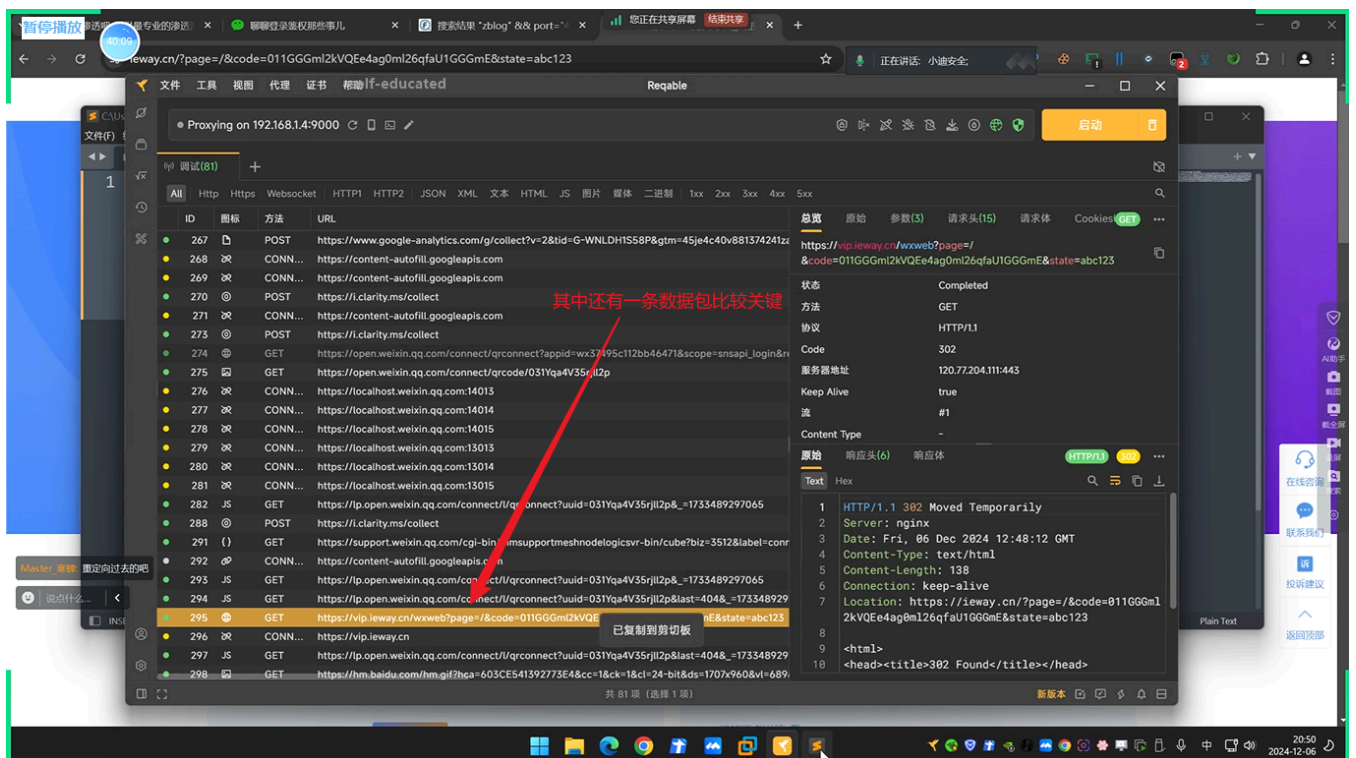
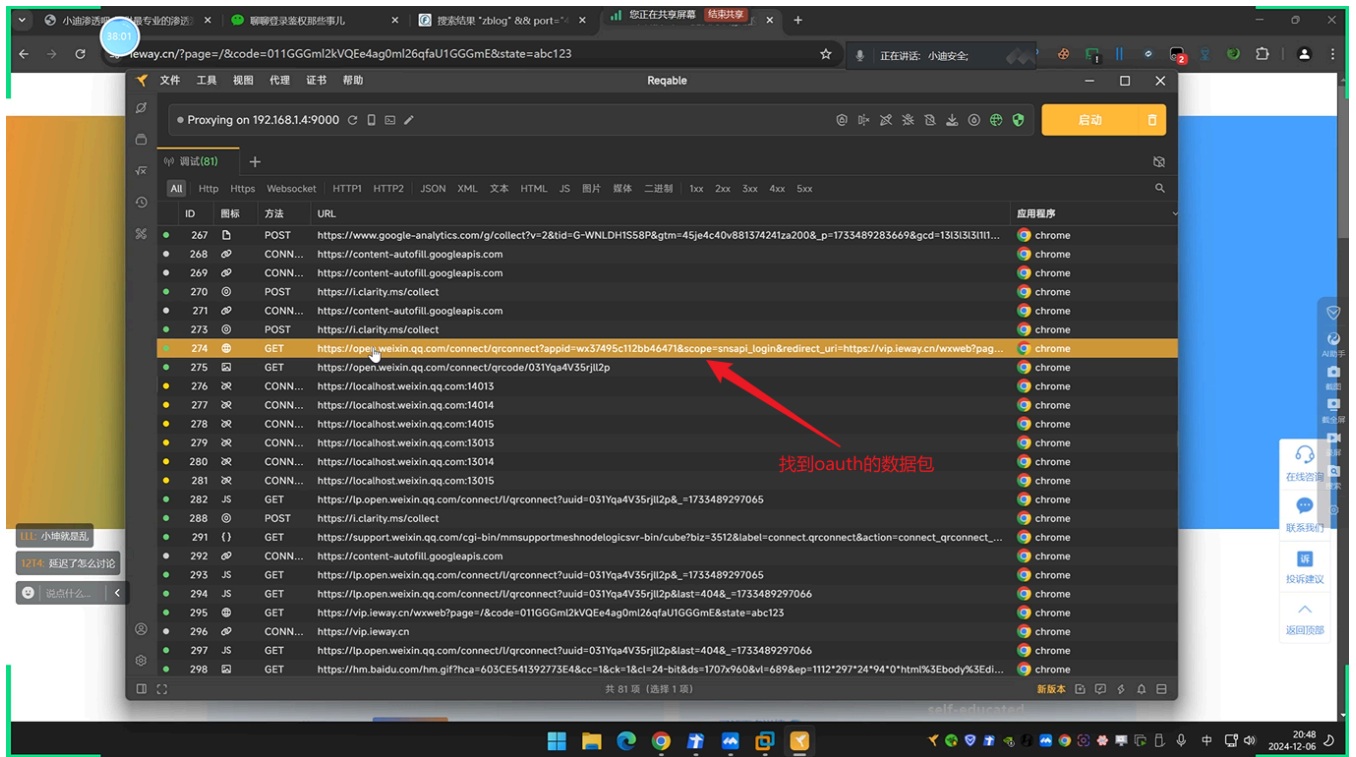
(G) 授权服务器核对授权码，向客户端发送访问令牌（access token）和更新令牌（refresh token）

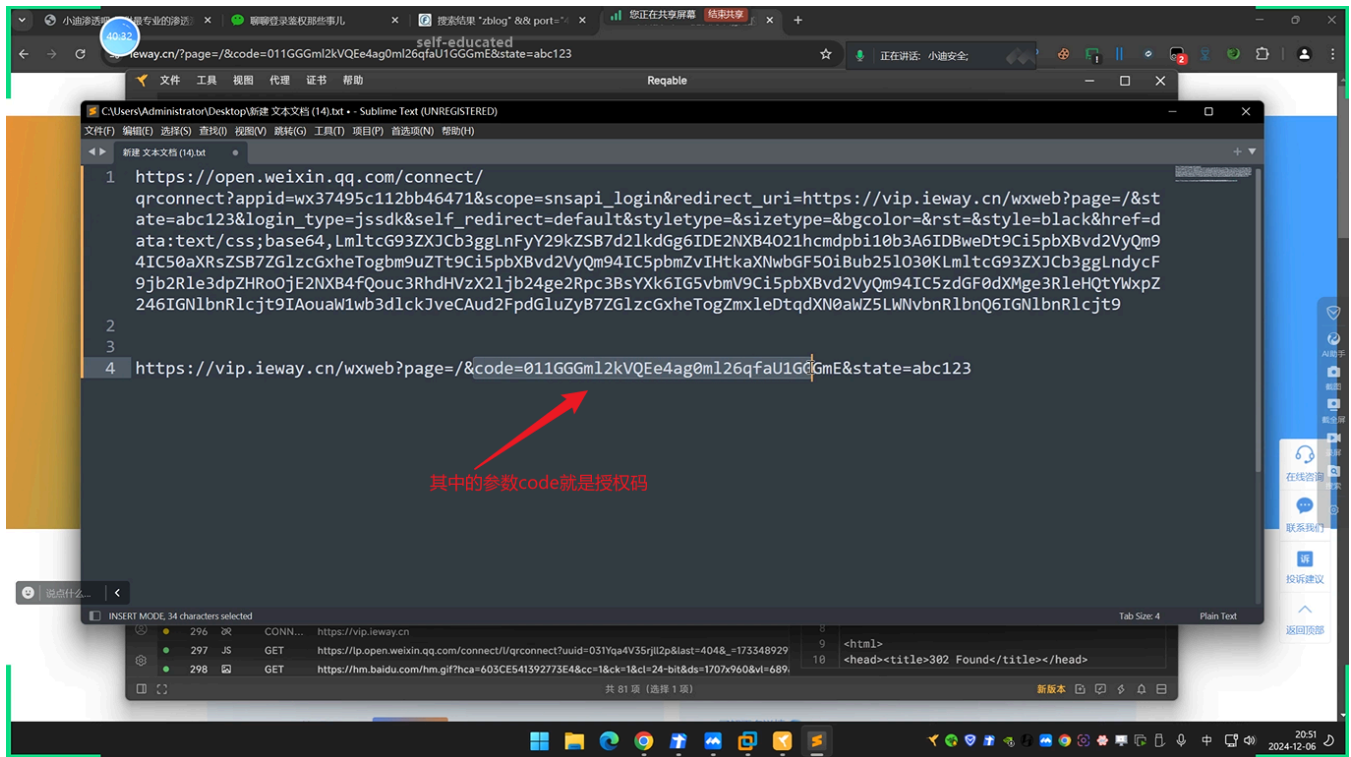
(H) 通过访问令牌向资源服务器请求资源 (I) 把访问令牌交给授权服务器进行校验

(3) 案例分析：







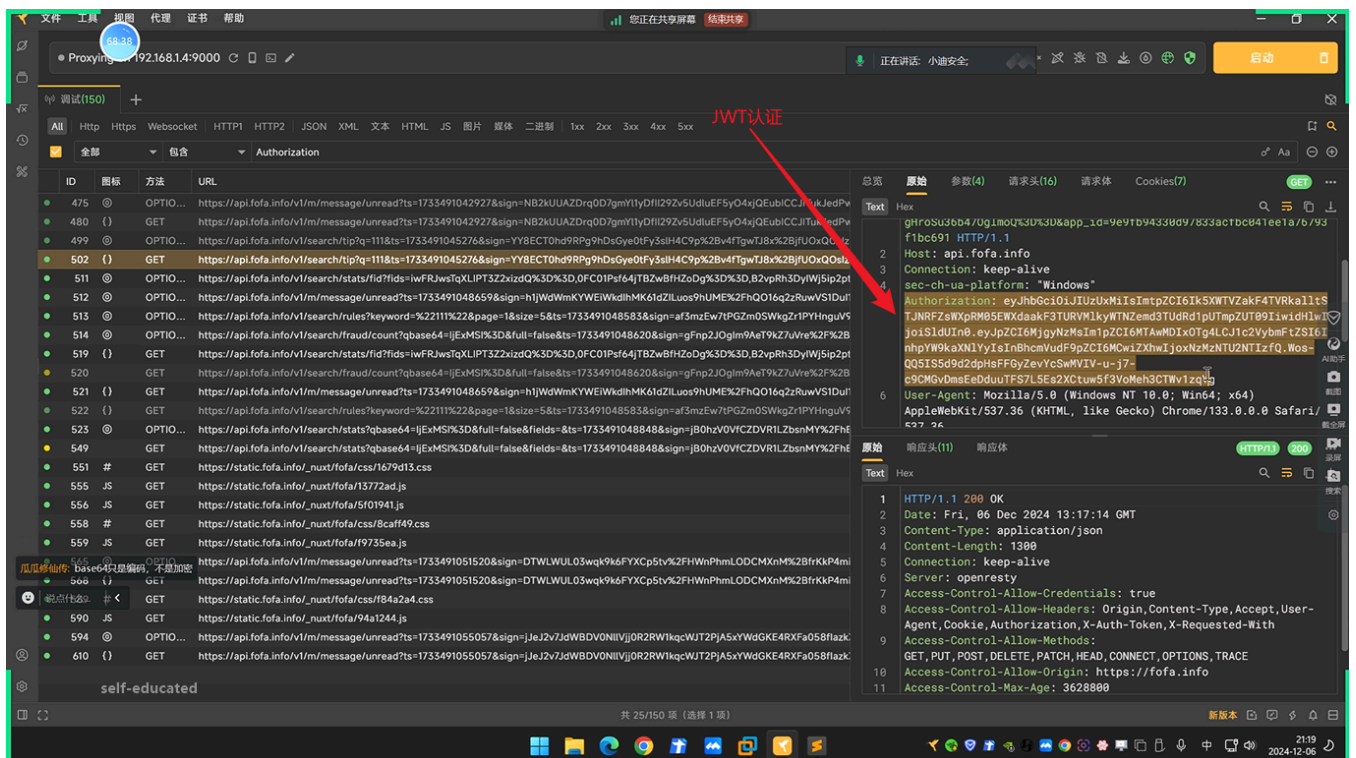
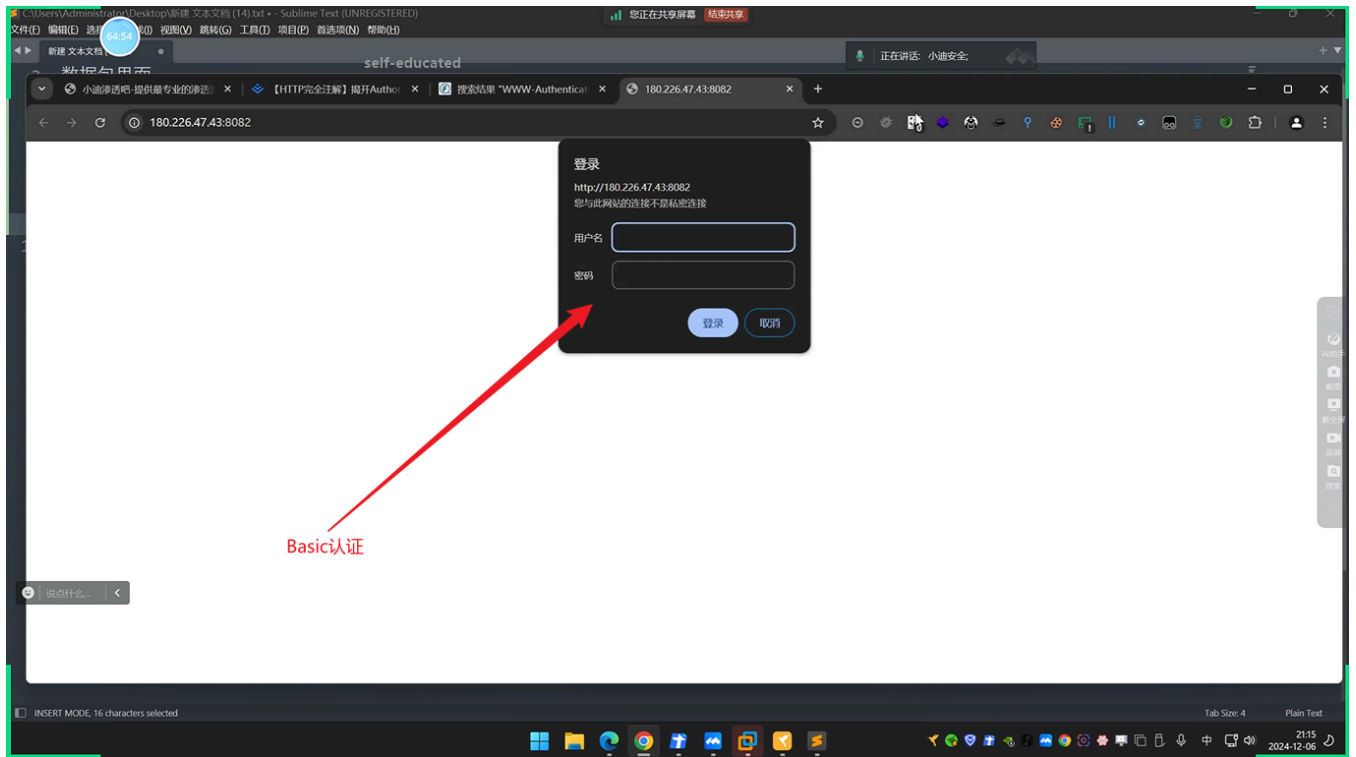


(4) 安全漏洞问题:

- 安全漏洞问题:
- 1、redirect_url 校验不严格导致code被劫持到恶意网站 (fuzz各种bypass方式)
- 2、client_id与redirect_url 不一致造成滥用劫持
- 3、A应用生成的code可以用在B应用上
- 4、state未设置csrf防护, 导致csrf风险
- 5、scope提权, 将低scope权限的code用于高权限场景
- 6、HTTP劫持, 网络层中间人攻击
- 7、点击劫持: 通过点击劫持, 恶意网站会在以下位置加载目标网站: 透明 iFrame (参见 [iFrame]) 覆盖在一组虚拟的顶部 精心构造的按钮直接放置在 目标站点上的重要按钮。当用户单击可见的 按钮, 他们实际上是在单击一个按钮 (例如“授权” 按钮) 在隐藏页面上。

3.2.5 Authorization

- #Authorization头:
- 参考: <https://juejin.cn/post/7300812626279251987>
- 授权方案:
- 1、Basic认证
- 2、Digest认证
- 3、Bearer认证
- 4、JWT认证
- 5、API密钥认证
- 6、双因素认证
- 7、其他一些认证方式
- 安全漏洞问题:
- JWT攻防, Token劫持等



3.3 总结



- 1 安全影响:
- 1、通过以上学习了解到http/https差异（抓包）
- 2、身份鉴权测试中要区别oauth,jwt,token差异